INDEX

SR. NO	TITLE	PAGE NO.
1.	Program to create, display and modify a matrix	
2.	Programs on arithmetic operations on matrix	
3.	Program to solve systems of linear equations	
4.	Program to generate elementary, continuous and discrete signals	
5.	Program on basic operations on continuous and discrete signals	
6.	Program to find linear convolution of two sequences	
7.	Given the input signal, program to find the response of a system	
8.	For a given network circuit, find the impulse response and unit step response of a system	
9.	Program to perform verification of properties of convolution sum	
10.	Program to compute frequency response of a system	
11.	Programs to find Z-transform and inverse Z-transform of a sequence. Simulate pole zero	
12.	Program to solve difference equations (upto 2 nd order)	

1. Program to create, display and modify a matrix.

CODE

```
clc;
close all;
clear all;
A=[1 1 -2;2 -1 1;3 1 -1]
B=[1 1 1;2 5 7;2 1 -1]
MATRIXADDITION=A+B
MATRIXSUBTRACTION=A-B
MATRIXMULTIPLICATION=A*B
ELEMENTWISEMULTIPLICATION=A.*B
RIGHTDIVISION=A/B
LEFTDIVISION=A\B
ELEMENTWISERIGHTDIVISION=A./B
ELEMENTWISELEFTDIVISION=A.\B
INVERSE=inv(A)
EXPONENTOFMATRIX=A^2
ELEMENTWISEEXPONENTOFMATRIX=A.^B
TRANSPOSE=A'
ARRAYTRANSPOSE=A.'
```

OUTPUT

```
A =

     1     1    -2
     2    -1     1
     3     1    -1


B =

     1     1     1
     2     5     7
     2     1    -1


MATRIXADDITION =

     2     2    -1
     4     4     8
     5     2    -2


MATRIXSUBTRACTION =

     0     0    -3
     0    -6    -6
     1     0     0
```

```
MATRIXMULTIPLICATION =

    -1     4    10
     2    -2    -6
     3     7    11


ELEMENTWISEMULTIPLICATION =

     1     1    -2
     4    -5     7
     6     1     1


RIGHTDIVISION =

   -5.0000    0.7500    2.2500
   12.0000   -2.0000   -3.0000
    3.0000   -0.5000    0.5000


LEFTDIVISION =

    0.8000    1.2000    1.2000
   -1.0000   -5.0000   -9.0000
   -0.6000   -2.4000   -4.4000
```

```
ELEMENTWISERIGHTDIVISION =

    1.0000    1.0000   -2.0000
    1.0000   -0.2000    0.1429
    1.5000    1.0000    1.0000


ELEMENTWISELEFTDIVISION =

    1.0000    1.0000   -0.5000
    1.0000   -5.0000    7.0000
    0.6667    1.0000    1.0000


INVERSE =

         0    0.2000    0.2000
   -1.0000   -1.0000    1.0000
   -1.0000   -0.4000    0.6000


EXPONENTOFMATRIX =

    -3    -2     1
     3     4    -6
     2     1    -4
```

```
EXPONENTOFMATRIX =

    -3    -2     1
     3     4    -6
     2     1    -4


ELEMENTWISEEXPONENTOFMATRIX =

     1     1    -2
     4    -1     1
     9     1    -1


TRANSPOSE =

     1     2     3
     1    -1     1
    -2     1    -1


ARRAYTRANSPOSE =

     1     2     3
     1    -1     1
    -2     1    -1
```

2. Programs on arithmetic operations on matrix.

CODE

```
A = input('Enter number of rows: ');
B = input('Enter number of columns: ');
for i = 1:A
for j = 1:B
    str = ['Enter element in row ' num2str(i) ', col ' num2str(j) ': '];
    T(i,j) = input(str);
end
end
%C = input('Enter number of rows: ');
%D = input('Enter number of columns: ');
strn = ['Enter elements for matrix S']
for i = 1:A
 for j = 1:B
     strn = ['Enter element in row ' num2str(i) ', col ' num2str(j) ': '];
     S(i,j) = input(strn);
 end
end
string= ['Addition is:']
for i=1:A
 for j=1:B
     E(i,j)=0;
 end
end
E(i,j)=0
for i= 1:m
 for j=1:B
    E(i,j)= T(i,j)+ S(i,j)
 end
End
```

OUTPUT

```
>> matrixaddition
Enter number of rows:
2
Enter number of columns:
2
Enter element in row 1, col 1:
1
Enter element in row 1, col 2:
2
Enter element in row 2, col 1:
3
Enter element in row 2, col 2:
4

strn =

    'Enter elements for matrix S'

Enter element in row 1, col 1:
4
Enter element in row 1, col 2:
3
Enter element in row 2, col 1:
2
Enter element in row 2, col 2:
1
```

```
E =

     5     5
     5     5
```

3. Program to solve systems of linear equations.

CODE

```
clc;
close all;
syms x y z
eqn1= 2*x+y+z==2;
eqn2=-x+y-z==3;
eqn3=x+2*y+3*z==-10;
[A,B] = equationsToMatrix([eqn1,eqn2,eqn3],[x,y,z]);
X=linsolve(A,B);
disp(X)
```

OUTPUT

3x1 sym

	1
1	3
2	1
3	-5

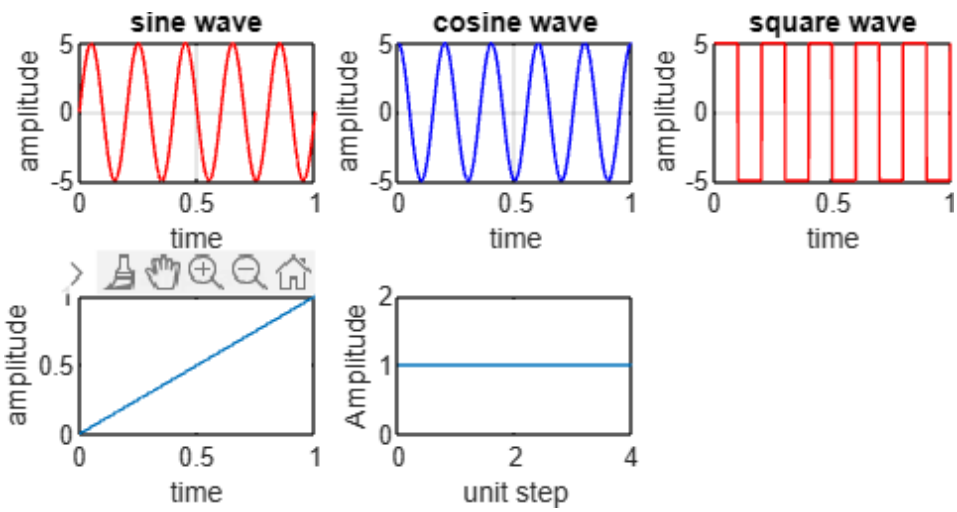
4. Program to generate elementary, continuous and discrete signals.

CODE

```
clc;
clear;
close all;
t=0:0.001:1;
f=input("enter the value of frequency");
a=input("enter the value of amplitude");
subplot(3,3,1);
y=a*sin(2*pi*f*t);
plot(t,y,'r');
xlabel('time');
ylabel('amplitude');
title('sine wave');
grid on;
subplot(3,3,2);
z=a*cos(2*pi*f*t);
plot(t,z,'b');
xlabel('time');
ylabel('amplitude');
title('cosine wave');
grid on;
subplot(3,3,3);
s=a*square(2*pi*f*t);
plot(t,s,'r');
xlabel('time');
ylabel('amplitude');
title('square wave');
grid on;
subplot(3,3,4);
plot(t,t);
xlabel('time');
ylabel('amplitude');
title('rampwave');
n=input('Enter n values');
t=0:1:n-1;
y1=ones(1,n);
subplot(3,3,5);
plot(t,y1);
ylabel('Amplitude');
xlabel('unit step');
```

OUTPUT

```
enter the value of frequency
5
enter the value of amplitude
5
Enter n values
5
```



5. Program on basic operations on continuous and discrete signals

CODE

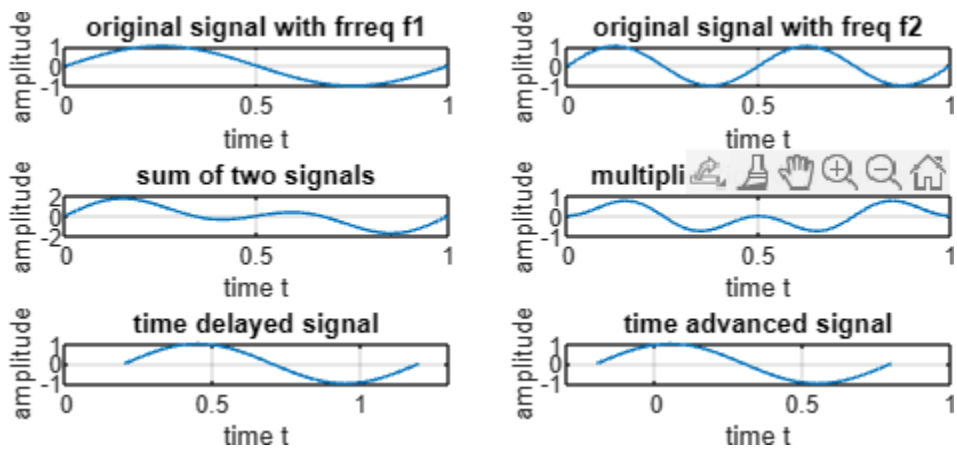
```
clc;
t=0:0.025:1;
a=1; f1=1; f2=2;
s1=a*sin(2*pi*f1*t);
s2=a*sin(2*pi*f2*t);
fprintf('\n 1. operation on continous time signal');
fprintf('\n 2. operation on discrete time signal\n');
ch=input('\n\n enter your choice');
switch ch
   case 1
       figure
       subplot(5,2,1);
       plot(t,s1);
       grid;
       title('original signal with frreq f1');
       xlabel('time t');
       ylabel('amplitude');
       subplot(5,2,2);
       plot(t,s2);
       grid;
       title('original signal with freq f2');
       xlabel('time  t');
       ylabel('amplitude');
       subplot(5,2,3);
       plot(t,s1+s2);
       grid;
       xlabel('time t');
       ylabel('amplitude');
       title('sum of two signals');
       subplot(5,2,4);
       plot(t,s1.*s2);
       grid;
       xlabel('time t');
       ylabel('amplitude');
       title('multiplication of s1 and s2');
       p=t+0.2;
       subplot(5,2,5);
       plot(p,s1);
       axis([0 1.3 -1 1]);
       grid;
       xlabel('time t');
       ylabel('amplitude');
       title('time delayed signal');
       p=t-0.2;
```

```
    subplot(5,2,6);
    plot(p,s1);
    axis([-0.3 1 -1 1]);
    grid;
    xlabel('time t');
    ylabel('amplitude');
    title('time advanced signal');
case 2
    n=-10:1:10;
    l=length(n);
    f1=0.1;
    f2=0.125;
    x1=sin(2*pi*f1*n);
    x2=sin(2*pi*f2*n);
    figure;
    subplot(4,2,1);
    stem(n,x1);
    title('discrete sine wave x1(n) with time period=10');
    grid on;
    subplot(4,2,2);
    stem(n,x2);
    title(' discrete sine wave x2(n) with time period=8');
    grid on;
    x3=x1+x2;
    subplot(4,2,3);
    stem(n,x3);
    title('sum of two discrete time signals');
    grid on;
    x4=x1.*x2;
    subplot(4,2,4);
    stem(n,x4);
    title('multiplication of two discrete time signals');
    grid on;
    no=2;
    x5=sin(2*pi*f1*(n-no));
    subplot(4,2,5);
    stem(n,x5);
    title(' time shifted signal of x1(n)');
    grid on;x6=fliplr(x1);
    subplot(4,2,6);
    stem(n,x6);
    title(' time folded signal of x1(n)');
    grid on;
    x7=x2.*2;
    subplot(4,2,[7 8]);
    stem(n,x7);
    title(' amplitude scaled discrete time signal of x2(n) ');
    grid on;
    e=sum(abs(x7).^2);e
```

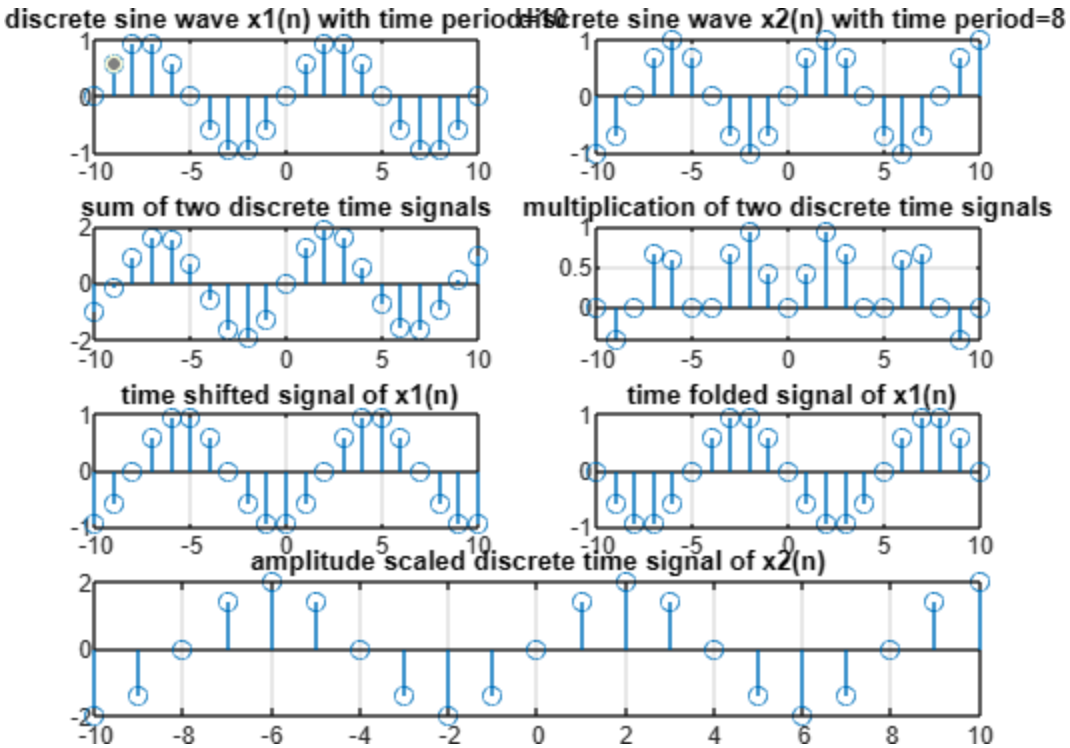
```
        l=length(n);
        p=e/l;
        p
End
```

OUTPUT

```
 1. operation on continous time signal
 2. operation on discrete time signal


 enter your choice
1
```




```
 1. operation on continous time signal
 2. operation on discrete time signal


 enter your choice
 2
```


discrete sine wave x1(n) with time period=10
discrete sine wave x2(n) with time period=8
sum of two discrete time signals
multiplication of two discrete time signals
time shifted signal of x1(n)
time folded signal of x1(n)
amplitude scaled discrete time signal of x2(n)

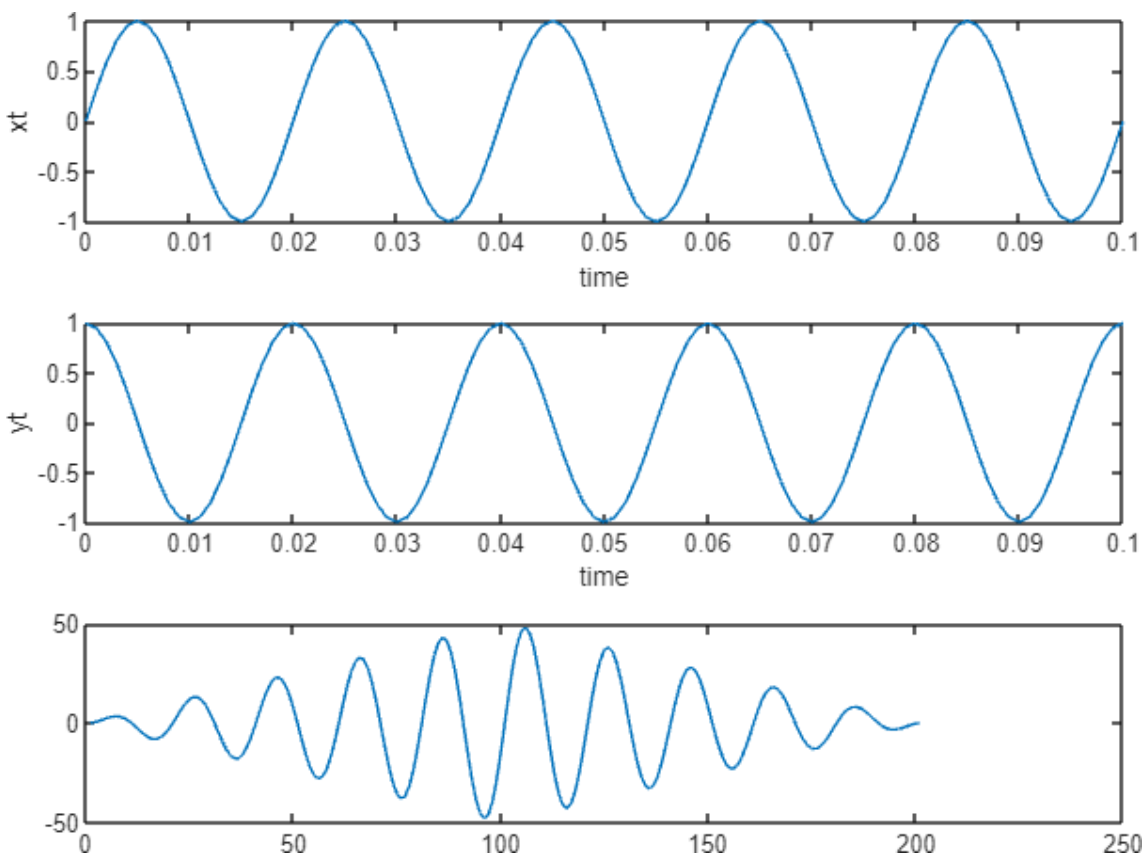
6. Program to find linear convolution of two sequences.

CODE

```
clc;
clear;
n1=input('enter the initial time value of i/p sequence');
x=input('enter the i/p sequence ');
n2=input('enter the initial time value of impulse response');
h=input('enter the impulse response');
sx=length(x);
sh=length(h);
nx=n1:sx+n1-1;
nh=n2:sh+n2-1;
ny=n1+n2:1:sx+sh+n1+n2-2;
y=conv(x,h);
figure(1);
subplot(2,2,1);
stem(nx,x);
xlabel('discrete time n');
ylabel('x(n)');
title('input signal');
subplot(2,2,2);
stem(nh,h);
xlabel('discrete time n');
ylabel('h(n)');
title('impulse response');
subplot(2,2,[3,4]);
stem(ny,y);
xlabel('discrete time n');
ylabel('y(n)');
title('linear convolution');
t=0:0.001:.1;
xt=sin(2*pi*50*t);
figure(2);
subplot(3,1,1);
plot(t,xt);
xlabel('time');
ylabel('xt');
yt=cos(2*pi*50*t);
subplot(3,1,2);
plot(t,yt);
xlabel('time');
ylabel('yt');
zt=conv(xt,yt);
subplot(3,1,3);
plot(zt);
```

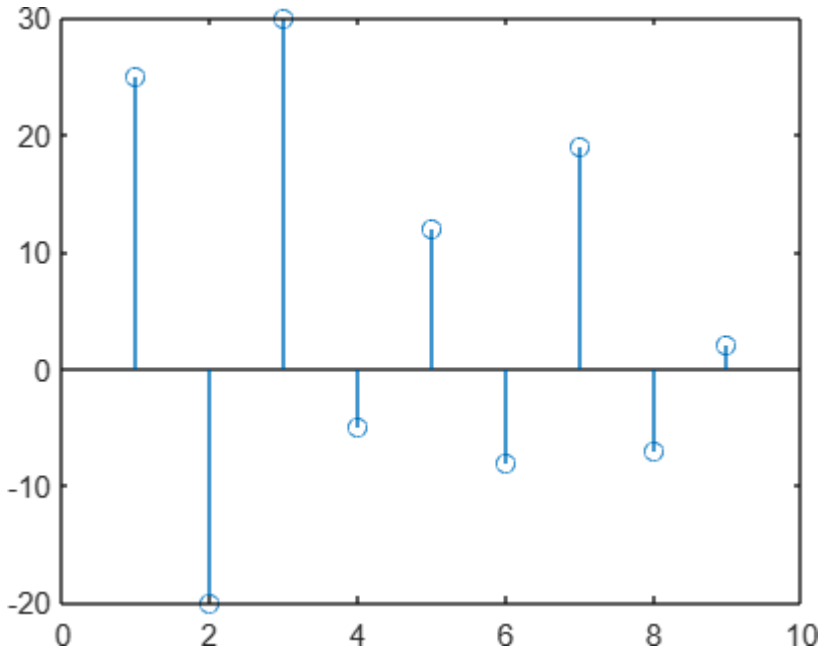
OUTPUT

```
enter the initial time value of i/p sequence
0.1
enter the i/p sequence
4
enter the initial time value of impulse response
5
enter the impulse response
5
```



7. Given the input signal, program to find the response of a system.

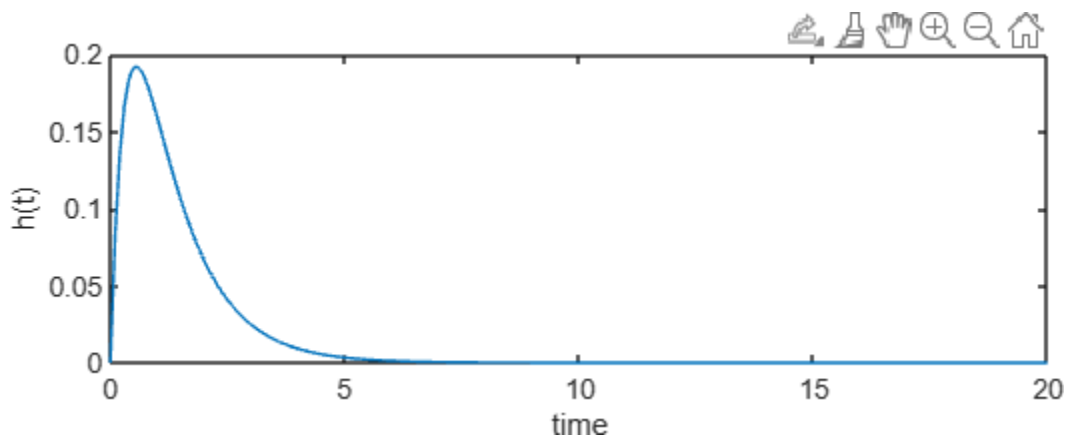
```
clc;
clear all;
close all;
%y[n]=-y[n-1]+5x[n]+2x[n-3]
b=[5 0 0 2]; %5x(n),2x(n-3)
a=[1 1]; %y(n)+y(n-1)
% from an input x[n]=[5,1,2,3,1,0,1,2,-1]
x=[5,1,2,3,1,0,1,2,-1];
y=filter(b,a,x)
stem(y)
```



8. For a given network circuit, find the impulse response and unit step response of a system.

CODE

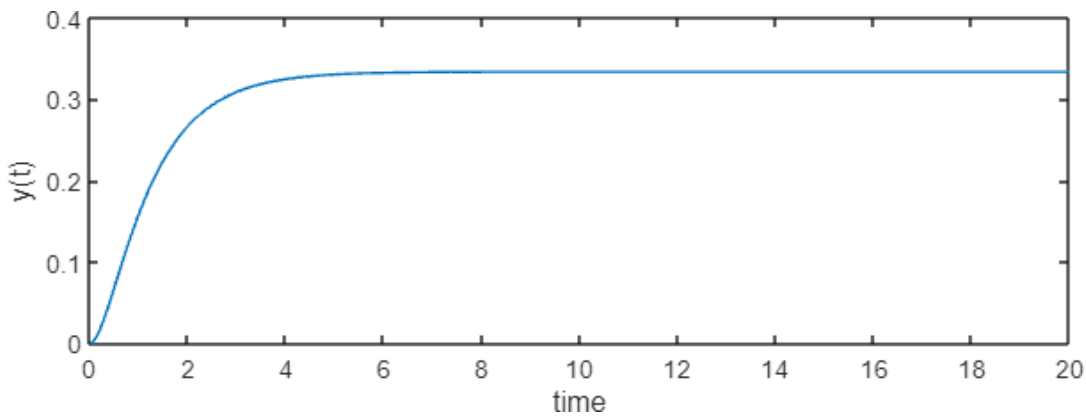
```
clc;
clear all;
close all;
syms s complex;
h=1/((s^2)+4*s+3);
disp('Impulse response of the system n/t is ');
h=ilaplace(h);
simplify(h);
disp(h);
t=0:0.1:20;
h1=subs(h,t);
subplot(2,1,1);
plot(t,h1);
xlabel('time');
ylabel('h(t)');
```




```
%STEP RESPONSE
clc;
clear all;
close all;
syms s complex;
y=1/(s*((s^2)+4*s+3));
disp('step response of the system y(t) is')
y=ilaplace(y);
simplify(y);
disp(y);
t=0:0.01:20;
y1=subs(y,t);
subplot(2,1,1);
plot(t,y1);
xlabel('time');
```

```
ylabel('y(t)');
```

OUTPUT



9. Program to perform verification of properties of convolution sum.

CODE

```
clc;
clear;
close all;
% commutative property
x=[1 2 3]
h=[1 2 3 4]
y_lhs=conv(x,h)
y_rhs=conv(h,x)
if y_lhs==y_rhs
   disp('commutative property is verified');
else
   disp('commutative property is not verified');
end
% associative property
x=[1 2 3]
h=[1 2 3 4]
g=[1 2 3]
y_lhs=conv(x,conv(h,g))
y_rhs=conv(conv(h,x),g)
if y_lhs==y_rhs
    disp('associative property is verified');
else
    disp('associative property is not verified');
end
% distributive property
x=[1 2 3]
h=[1 2 3 4]
g=[1 2 3 5]
y_lhs=conv(x,(h+g))
y_rhs=(conv(x,h)+conv(x,g))
if y_lhs==y_rhs
    disp('distributive property is verified');
else
    disp('distributive property is not verified');
end
```

OUTPUT

```
x =
     1     2     3

h =
     1     2     3     4

y_lhs =
     1     4    10    16    17    12

y_rhs =
     1     4    10    16    17    12
commutative property is verified
```

```
x =
     1     2     3

h =
     1     2     3     4

g =
     1     2     3

y_lhs =
     1     6    21    48    79    94    75    36

y_rhs =
     1     6    21    48    79    94    75    36
associative property is verified
```

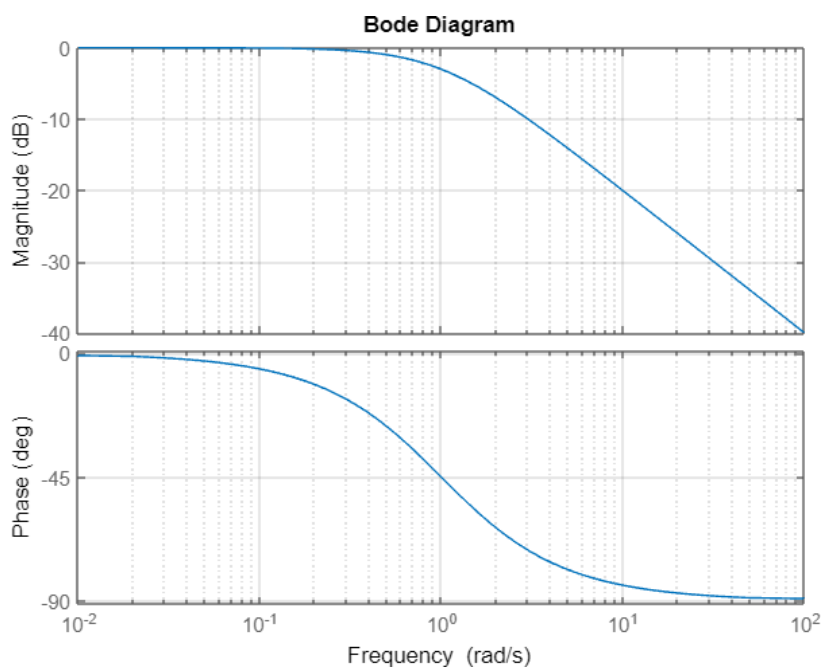
```
x =
     1     2     3

h =
     1     2     3     4

g =
     1     2     3     5

y_lhs =
     2     8    20    33    36    27

y_rhs =
     2     8    20    33    36    27
distributive property is verified
>>
```

10. Program to compute frequency response of a system.

CODE

```
clear
clc
close all
% Define Transfer function
num = [1];
den = [1, 1];
H = tf(num, den)
% Frequency Response
bode(H);
grid on
% Get Freqquency Response Data
wlist = [0.01, 0.1, 1, 2 ,3 ,5 ,10, 100];
[mag, phase, w] = bode(H, wlist);
for i=1:length(w)
magw(i) = mag(1,1,i);
phasew(i) = phase(1,1,i);
end
%magdB = 20*log10(magw); % Convert to dB
freq_data = [wlist; magw; phasew]'
```

OUTPUT

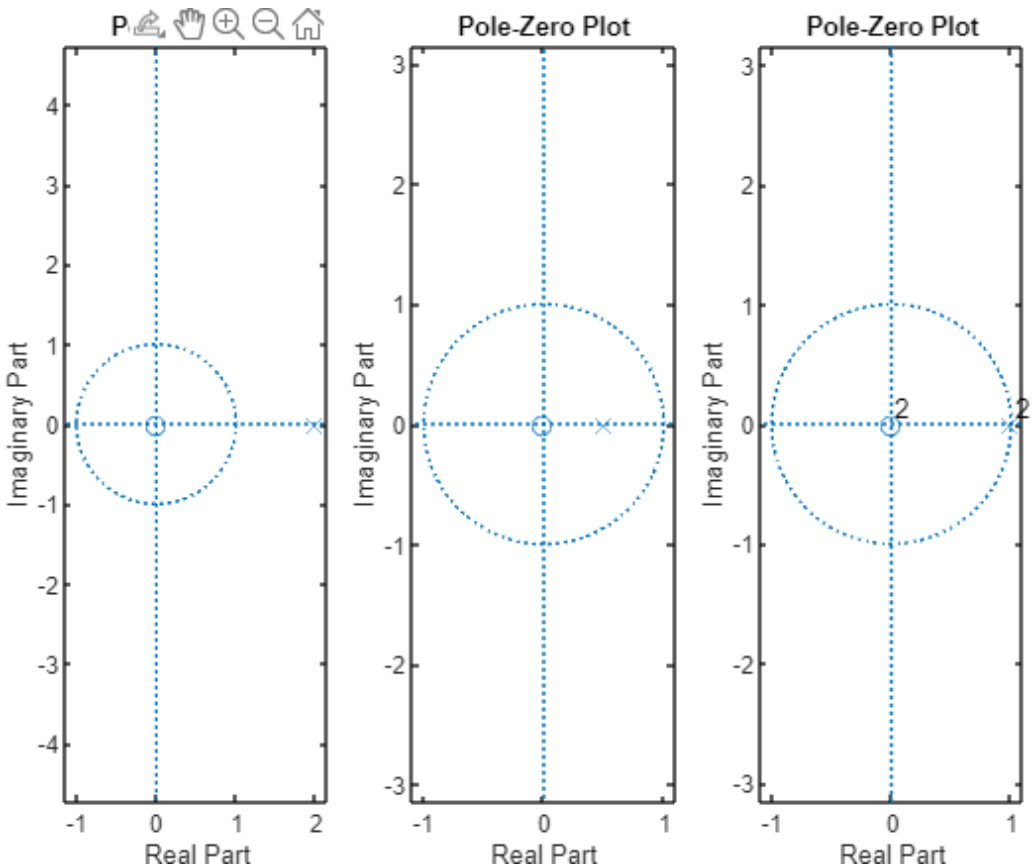


11. Programs to find Z-transform and inverse Z-transform of a sequence. Simulate pole zero.

CODE

```
clc;
clear all;
close all;
syms n;
a = 2;
x = a^n;
x = ztrans(x); %finding z transform
disp('z transform of a^n a>1');
disp(x);
syms n;
a = 0.5;
x = a^n;
x1 = ztrans(x);
disp('z transform of a^n 0<a<1');
disp(x1);
syms n;
a =2;
x = 1+n;
x2 = ztrans(x);
disp('Z transform of 1+n');
disp(x2);
A = iztrans(x);
disp('inverse of z transform of a^n a>1');
disp(A);
B = iztrans(x1);
disp('inverse of z transform of a^n 0<a<1');
disp(B);
c = iztrans(x2);
disp('inverse of z transform of 1+n');
disp(c);
subplot(1,3,1);
zplane([1 0],[1 -2]);
subplot(1,3,2);
zplane([1 0],[1 -2]);
subplot(1,3,2)
zplane([1 0],[1 -1/2]);
subplot(1,3,3);
zplane([1 0 0],[1 -2 1]);
```

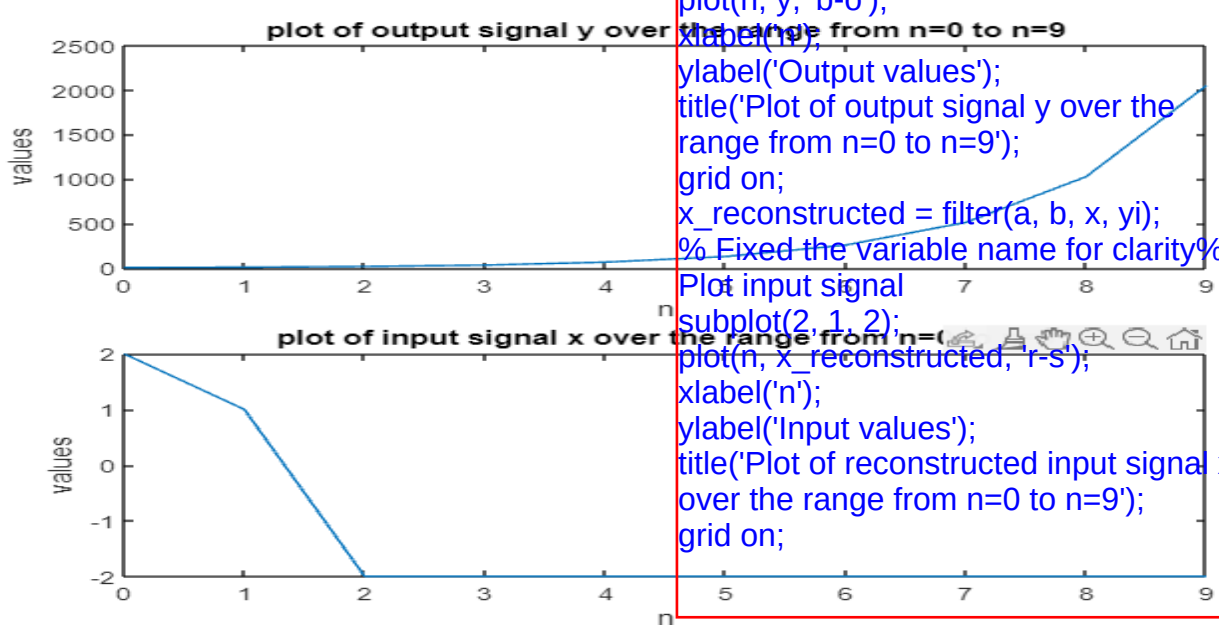
OUTPUT:



12. Program to solve difference equations (upto 2 nd order).

CODE

```
clc;
clear;
close all;
n=0:9;
a=[1 -1 -2];
b=[1 0 0];
yi=[1 1];
xi=0;
x=1;
zi=filtic(b,a,yi,xi)
y=filter(b,a,ones(1,10),zi)
x=filter(a,b,ones(1,10),yi)
subplot(2,1,1)
plot(n,y)
xlabel('n')
ylabel('values')
title('plot of output signal y over the range from n=0 to n=9')
subplot(2,1,2)
plot(n,x)
xlabel('n')
ylabel('values')
title('plot of input signal x over the range from n=0 to n=9')
```

```
clc;
clear;
close all;
% Define time range
n = 0:9;
% Define coefficients of the system's
difference equation
a = [1 -1 -2];
% b represents the numerator
(feedforward coefficients)
b = [1 0 0];
% Define initial conditions for output (yi)
and input (xi)
yi = [1 1];
xi = 0;

x = ones(1, length(n));
% Compute initial conditions for filter
zi = filtic(b, a, yi, xi);
% Correct usage of filtic function%
Compute output signal using the filter
y = filter(b, a, x, zi);
% Apply the filter with initial conditions%
Plot output signal
subplot(2, 1, 1);
plot(n, y, 'b-o');
xlabel('n');
ylabel('Output values');
title('Plot of output signal y over the
range from n=0 to n=9');
grid on;
x_reconstructed = filter(a, b, x, yi);
% Fixed the variable name for clarity%
Plot input signal
subplot(2, 1, 2);
plot(n, x_reconstructed, 'r-s');
xlabel('n');
ylabel('Input values');
title('Plot of reconstructed input signal x
over the range from n=0 to n=9');
grid on;
```

OUTPUT